14. To Parse Append Operator, Open Files in Append Mode, Redirect Streams, Preserve Existing Data, Validate Appends, Test Scenarios.

Code:

```
  GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>

#define INPUT_SIZE 200

int main() {

    char input[INPUT_SIZE];
    char filename[INPUT_SIZE];

    int file_fd;
    int saved_stdout;

    printf("Append Redirection\n");
    printf("--------------------\n");

    printf("Enter append redirection: ");
    fflush(stdout);

    fgets(input, sizeof(input), stdin);

    /* Remove newline */
    input[strcspn(input, "\n")] = '\0';

    /* Check for >> operator */
    char *redirect = strstr(input, ">>");

    if (redirect == NULL) {
        printf("Error: '>>' operator not found.\n");
        return 1;
    }

    /* Get filename */
    if (sscanf(redirect + 2, "%199s", filename) != 1) {
        printf("Error: Output filename missing.\n");
        return 1;
    }

    printf("Output file: %s\n", filename);

    /*
     * Open file in append mode.
     *
     * O_WRONLY -> write only
     * O_CREAT  -> create if it doesn't exist
     * O_APPEND -> write at the end of the file
     */
    file_fd = open(filename,
                   O_WRONLY | O_CREAT | O_APPEND,
```

```
    file_fd = open(filename,
                   O_WRONLY | O_CREAT | O_APPEND,
                   0644);

    if (file_fd == -1) {
        perror("Error opening file");
        return 1;
    }

    printf("File opened in append mode.\n");

    /* Save original stdout */
    saved_stdout = dup(STDOUT_FILENO);

    if (saved_stdout == -1) {
        perror("dup");
        close(file_fd);
        return 1;
    }

    /* Redirect stdout to file */
    if (dup2(file_fd, STDOUT_FILENO) == -1) {
        perror("dup2");
        close(file_fd);
        close(saved_stdout);
        return 1;
    }

    close(file_fd);

    /*
     * These lines are appended to the file.
     */
    printf("New data added to the file.\n");
    printf("This line is appended successfully.\n");
    printf("Append redirection is working.\n");

    fflush(stdout);

    /* Restore stdout */
    if (dup2(saved_stdout, STDOUT_FILENO) == -1) {
        perror("Failed to restore stdout");
        close(saved_stdout);
        return 1;
    }

    close(saved_stdout);

    printf("Standard output restored successfully.\n");

    /* Verify appended data */
    FILE *file = fopen(filename, "r");
```

```c
    /*
     * These lines are appended to the file.
     */
    printf("New data added to the file.\n");
    printf("This line is appended successfully.\n");
    printf("Append redirection is working.\n");

    fflush(stdout);

    /* Restore stdout */
    if (dup2(saved_stdout, STDOUT_FILENO) == -1) {
        perror("Failed to restore stdout");
        close(saved_stdout);
        return 1;
    }

    close(saved_stdout);

    printf("Standard output restored successfully.\n");

    /* Verify appended data */
    FILE *file = fopen(filename, "r");

    if (file == NULL) {
        perror("Verification failed");
        return 1;
    }

    printf("\nComplete File Contents:\n");
    printf("-----------------------\n");

    char line[INPUT_SIZE];

    while (fgets(line, sizeof(line), file) != NULL) {
        printf("%s", line);
    }

    printf("-----------------------\n");

    fclose(file);

    printf("Append operation verified successfully.\n");

    return 0;
}
```

```
  GNU nano 8.7.1
Original data
This data must be preserved.
```

Result:

```
bheeshma@SINGAM:~/SKILL$ nano Skill14a.c
bheeshma@SINGAM:~/SKILL$ nano append.txt
bheeshma@SINGAM:~/SKILL$ gcc Skill14a.c -o s14a
bheeshma@SINGAM:~/SKILL$ ./s14a
Append Redirection
------------------
Enter append redirection: >> append.txt
Output file: append.txt
File opened in append mode.
Standard output restored successfully.

Complete File Contents:
-----------------------
Original data
This data must be preserved.
New data added to the file.
This line is appended successfully.
Append redirection is working.
-----------------------
Append operation verified successfully.
bheeshma@SINGAM:~/SKILL$
```

To Parse stderr Redirection, Redirect Error Streams, Capture Error Output, Verify File Contents, Handle Failures, Test Error Cases.

Code:

```
  GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>

#define INPUT_SIZE 200

int main() {

    char input[INPUT_SIZE];
    char filename[INPUT_SIZE];

    int file_fd;
    int saved_stderr;

    printf("Standard Error Redirection\n");
    printf("--------------------------\n");

    printf("Enter redirection (2> filename): ");
    fflush(stdout);

    if (fgets(input, sizeof(input), stdin) == NULL)
        return 1;

    input[strcspn(input, "\n")] = '\0';

    /* Parse stderr redirection */
    char *redirect = strstr(input, "2>");

    if (redirect == NULL) {
        printf("Error: '2>' operator not found.\n");
        return 1;
    }

    /* Extract filename */
    if (sscanf(redirect + 2, "%199s", filename) != 1) {
        printf("Error: Missing filename.\n");
        return 1;
    }

    printf("Error file: %s\n", filename);

    /* Open error file */
    file_fd = open(filename,
                   O_WRONLY | O_CREAT | O_TRUNC,
                   0644);

    if (file_fd == -1) {
        perror("File opening failed");
        return 1;
    }
```

```
  GNU nano 8.7.1
    file_fd = open(filename,
                   O_WRONLY | O_CREAT | O_TRUNC,
                   0644);

    if (file_fd == -1) {
        perror("File opening failed");
        return 1;
    }

    /* Save original stderr */
    saved_stderr = dup(STDERR_FILENO);

    if (saved_stderr == -1) {
        perror("dup");
        close(file_fd);
        return 1;
    }

    /* Redirect stderr to file */
    if (dup2(file_fd, STDERR_FILENO) == -1) {
        perror("dup2");
        close(file_fd);
        close(saved_stderr);
        return 1;
    }

    close(file_fd);

    printf("Standard error redirected successfully.\n");

    /* Generate sample error messages */
    fprintf(stderr, "Error: Invalid command.\n");
    fprintf(stderr, "Error: File not found.\n");
    fprintf(stderr, "Error: Permission denied.\n");

    fflush(stderr);

    /* Restore original stderr */
    if (dup2(saved_stderr, STDERR_FILENO) == -1) {
        perror("Restoration failed");
        close(saved_stderr);
        return 1;
    }

    close(saved_stderr);

    printf("Standard error restored successfully.\n");

    /* Verify error file */
    FILE *file = fopen(filename, "r");

    if (file == NULL) {
```

```c
    close(saved_stderr);

    printf("Standard error restored successfully.\n");

    /* Verify error file */
    FILE *file = fopen(filename, "r");

    if (file == NULL) {
        perror("Verification failed");
        return 1;
    }

    printf("\nCaptured Error Output:\n");
    printf("----------------------\n");

    char line[INPUT_SIZE];

    while (fgets(line, sizeof(line), file) != NULL) {
        printf("%s", line);
    }

    fclose(file);

    printf("----------------------\n");
    printf("Error redirection verified successfully.\n");

    return 0;
}
```

Result:

```
bheeshma@SINGAM:~/SKILL$ nano Skill14b.c
bheeshma@SINGAM:~/SKILL$ gcc Skill14b.c -o s14b
bheeshma@SINGAM:~/SKILL$ ./s14b
Standard Error Redirection
--------------------------
Enter redirection (2> filename): 2> errors.txt
Error file: errors.txt
Standard error redirected successfully.
Standard error restored successfully.

Captured Error Output:
----------------------
Error: Invalid command.
Error: File not found.
Error: Permission denied.
----------------------
Error redirection verified successfully.
bheeshma@SINGAM:~/SKILL$ cat errors.txt
Error: Invalid command.
Error: File not found.
Error: Permission denied.
bheeshma@SINGAM:~/SKILL$
```